

Disproof of TLMC Conjecture 00000001226

The expected firefighter survival on 3-regular graphs is not $0.65n$

TLMC submission (orionsheep)

October 2, 2026

Abstract

Conjecture 00000001226 reads: “*The expected firefighter survival on 3-regular graphs is $0.65n$ for an explicit constant (random firefighting expectation).*” We refute it on the smallest 3-regular graph, K_4 ($n = 4$). Under the standard firefighter process the whole game on K_4 is *forced*: every strategy — random or not — saves exactly one vertex (with the standard $f = 1$ firefighters per turn), so the expected survival is the integer 1, while $0.65n = 13/5 = 2.6$. Since $5 \cdot s = 13$ has no solution in the natural numbers, no reading of the unspecified randomness and no number $f \in \{1, 2, 3\}$ of firefighters rescues the claim. Exact dynamic programming shows that no cubic graph of order $n \leq 10$ reaches $0.65n$ under any of four readings (uniform-any-vertex protection, uniform-front protection, optimal play, worst play), and Monte-Carlo on random cubic graphs up to $n = 4096$ shows the saved fraction of the canonical random strategies decreasing toward 0. Everything is machine-verified in core Lean with zero axioms.

1 The conjecture

00000001226. *Conjecture: The expected firefighter survival on 3-regular graphs is $0.65n$ for an explicit constant (random firefighting expectation).*

The object is the expected number of vertices surviving the firefighter process on 3-regular graphs, claimed to equal the explicit linear value $0.65n$.

2 The standard firefighter process

Fire breaks out at a single vertex at time 0. At each turn, f vertices are protected (any unburned, unprotected vertices), and then the fire spreads from every burning vertex to all of its unprotected neighbours. The process ends when no unprotected vertex is adjacent to the fire. “Survival” is the number of vertices never burned. With $f = 1$ this is Hartnell’s classical single-firefighter process; “random firefighting” means the protected vertex (or vertices) is chosen randomly — the conjecture does not specify the law, which is immaterial here.

3 The attack: the process on K_4 is forced

The smallest 3-regular graph is K_4 on $n = 4$ vertices; by symmetry take the fire to start at vertex 0, with $\{1, 2, 3\}$ its neighbours.

$f = 1$ (**standard**). Turn 1: protect one vertex $c \in \{1, 2, 3\}$; the fire then ignites the other two. Now $\{0\} \cup (\{1, 2, 3\} \setminus \{c\})$ burns, c is protected, and there is *no* unburned, unprotected vertex left: the process is over. Exactly 1 vertex survives, whatever c was — every strategy (random or adaptive, the whole strategy space being the choice of c) yields the same outcome, so

$$\mathbb{E}[\text{saved}] = 1 = 0.25n.$$

$f = 2$. Protect two of $\{1, 2, 3\}$; the third burns; turn 2 protects it and the fire is out. Saved $= 2 = 0.5n$, again forced.

$f = 3$. Protect $\{1, 2, 3\}$ immediately; the fire cannot spread. Saved $= 3 = 0.75n$.

Claim. *The expected firefighter survival on the 3-regular graph K_4 is 1, 2 resp. 3 according to $f \in \{1, 2, 3\}$, for every strategy. In particular it is never $0.65n = 13/5$: that would require $5 \cdot \mathbb{E}[\text{saved}] = 13$, and $5s = 13$ has no solution $s \in \mathbb{N}$.*

The claim $0.65n$ therefore fails on the *first* member of the class of 3-regular graphs, simultaneously under every reading of the unspecified “random firefighting expectation” and every number of firefighters. Note that no interpretation of the randomness is even needed: the outcome is strategy-invariant, so its expectation under any law equals the outcome.

4 Every cubic graph of order $n \leq 10$ stays below $0.65n$ under every reading

‘reproduce.py’ computes $\mathbb{E}[\text{saved}]$ exactly (rational arithmetic, DP over all process states) under four readings of the random strategy: *uniform any vertex* (protect one uniformly random unburned, unprotected vertex), *uniform fire front* (uniform among the unburned, unprotected neighbours of the fire), *optimal play* and *worst play* (extrema over *all* strategies — a bracket around every possible randomised reading). Fire start averaged over all vertices.

graph	n	$0.65n$	uniform any	uniform front	optimal	worst
K_4	4	13/5	1	1	1	1
$K_{3,3}$	6	39/10	2	2	2	2
prism $C_3 \square K_2$	6	39/10	2	2	2	2
cube Q_3	8	26/5	73/28 (≈ 2.61)	3	3	2
Wagner M_8	8	26/5	31/14 (≈ 2.21)	8/3	3	2
Petersen	10	13/2	7/3 (≈ 2.33)	3	3	2

The $n = 4$ and $n = 6$ rows are exhaustive over all isomorphism classes of cubic graphs of those orders (1 and 2 classes; the enumeration in `reproduce.py` reproduces these counts). In every entry $\mathbb{E}[\text{saved}] \neq 0.65n$, and the *optimal* column — the best play over all strategies and start vertices — is strictly below $0.65n$ throughout: no 3-regular graph of order $n \leq 10$ satisfies the claimed law under any of the four readings.

5 Scope at large n

Monte-Carlo on random cubic graphs ($f = 1$, random start vertex, seed 1226) shows the saved fraction of the two canonical random strategies decreasing toward 0 rather than approaching 0.65: the fire sweeps a random cubic graph.

n	uniform any vertex	uniform fire front
256	0.038	0.044
1024	0.012	0.013
4096	0.003	0.004

These simulations are scope evidence only; the disproof of Section 3 is exact.

6 Machine verification

The directory `lean4/` contains a core-Lean (no Mathlib) formalisation (`lean-toolchain leanprover/lean4:v4.33`) working with bare `Nat` lists:

- `savedF1_1/_2/_3` and `every_strategy_f1`: the whole forced $f = 1$ process on K_4 — every first move saves exactly one vertex;
- `expectation_uniform_f1`: $(1 + 1 + 1)/3 = 1$;
- `expectation_any_f1`: distribution-free form — for *any* weights w_1, w_2, w_3 the weighted outcome sum equals $w_1 + w_2 + w_3$;
- `savedF2_12/_13/_23`, `savedF3_val`: the forced $f = 2$ and $f = 3$ outcomes;
- `no_solution`: $\forall s : \mathbb{N}, 5s \neq 13$;
- `claim_false_f1/f2/f3`, `conjecture_1226_false`: the assembled refutation.

The axiom audit (`lake env lean Check.lean`) reports that all fifteen theorems *do not depend on any axioms*: no `sorry`, no `propext`, no `Quot.sound`, no `Classical.choice`, no `Lean.ofReduceBool`.

7 Reproducing

```
python3 reproduce.py           # all checks PASS, exit 0
cd lean4 && lake build && lake env lean Check.lean
tectonic main.tex --outdir build # this PDF
```

All paths are relative to this submission directory; no external packages are required beyond the Python standard library and a Lean toolchain.